



Git it done!
Introducción al control de versiones con *git*

Mario Sánchez Prada
msanchez@igalia.com

A Coruña, 14 de Mayo de 2010

“I think Git overshadows any other achievement that Linus guy ever had. He should stop working on that hobby kernel, and put more time into Git, I think it has potential.”

*Eitan Isaacson
Open Source developer
October 21st, 2009*

- Ingeniero en Informática por la *UDC*
 - `www.fi.udc.es`
- Miembro de la empresa *Igalia*
 - `www.igalia.com`
 - `blogs.igalia.com/mario`
- *GNOME & Maemo* developer
 - `live.gnome.org/MarioSanchez`
 - `garage.maemo.org/users/msanchez`

Acerca de mí

- Ingeniero en Informática por la *UDC*
 - `www.fi.udc.es`
- Miembro de la empresa *Igalia*
 - `www.igalia.com`
 - `blogs.igalia.com/mario`
- *GNOME & Maemo* developer
 - `live.gnome.org/MarioSanchez`
 - `garage.maemo.org/users/msanchez`
- Usuario de *git*

- Sistema de Control de Versiones **Distribuido** (*DVCS*)
- Inicialmente escrito en C por Linus Torvalds
- Sustituto de *BitKeeper* en el desarrollo del kernel de Linux
- Ampliamente usado hoy en día (no sólo en proyectos)

`http://git-scm.com`

Conceptos básicos

El problema

Gestión de cambios en un fichero, proyecto...

El problema

Gestión de cambios en un fichero, proyecto...

- Opción 0: no hacer nada (versión única)

El problema

Gestión de cambios en un fichero, proyecto...

- Opción 0: no hacer nada (versión única)
- Opción 1: Sistemas rudimentarios (copias manuales)
 - Proyecto

El problema

Gestión de cambios en un fichero, proyecto...

- Opción 0: no hacer nada (versión única)
- Opción 1: Sistemas rudimentarios (copias manuales)
 - Proyecto
 - Proyecto.bckp

El problema

Gestión de cambios en un fichero, proyecto...

- Opción 0: no hacer nada (versión única)
- Opción 1: Sistemas rudimentarios (copias manuales)
 - Proyecto
 - Proyecto.bckp
 - Proyecto-20100514.bckp

El problema

Gestión de cambios en un fichero, proyecto...

- Opción 0: no hacer nada (versión única)
- Opción 1: Sistemas rudimentarios (copias manuales)
 - Proyecto
 - Proyecto.bckp
 - Proyecto-20100514.bckp
 - Proyecto-20100514.bckp2

Gestión de cambios en un fichero, proyecto...

- Opción 0: no hacer nada (versión única)
- Opción 1: Sistemas rudimentarios (copias manuales)
 - Proyecto
 - Proyecto.bckp
 - Proyecto-20100514.bckp
 - Proyecto-20100514.bckp2
 - Proyecto-20100514.bckp2-final

El problema

Gestión de cambios en un fichero, proyecto...

- Opción 0: no hacer nada (versión única)
- Opción 1: Sistemas rudimentarios (copias manuales)
 - Proyecto
 - Proyecto.bckp
 - Proyecto-20100514.bckp
 - Proyecto-20100514.bckp2
 - Proyecto-20100514.bckp2-final
 - Proyecto-20100514.bckp2-definitivo

Gestión de cambios en un fichero, proyecto...

- Opción 0: no hacer nada (versión única)
- Opción 1: Sistemas rudimentarios (copias manuales)
 - Proyecto
 - Proyecto.bckp
 - Proyecto-20100514.bckp
 - Proyecto-20100514.bckp2
 - Proyecto-20100514.bckp2-final
 - Proyecto-20100514.bckp2-definitivo
 - Proyecto-20100514.bckp2-definitivo-final

Gestión de cambios en un fichero, proyecto...

- Opción 0: no hacer nada (versión única)
- Opción 1: Sistemas rudimentarios (copias manuales)
 - Proyecto
 - Proyecto.bckp
 - Proyecto-20100514.bckp
 - Proyecto-20100514.bckp2
 - Proyecto-20100514.bckp2-final
 - Proyecto-20100514.bckp2-definitivo
 - Proyecto-20100514.bckp2-definitivo-final
 - Proyecto-????????

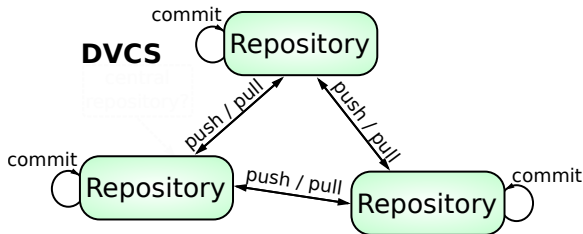
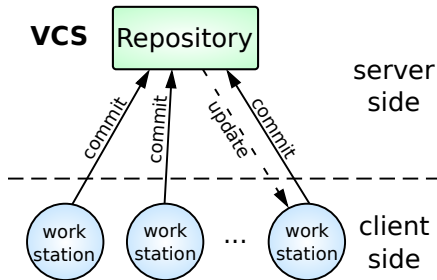
Gestión de cambios en un fichero, proyecto...

- Opción 0: no hacer nada (versión única)
- Opción 1: Sistemas rudimentarios (copias manuales)
 - Proyecto
 - Proyecto.bckp
 - Proyecto-20100514.bckp
 - Proyecto-20100514.bckp2
 - Proyecto-20100514.bckp2-final
 - Proyecto-20100514.bckp2-definitivo
 - Proyecto-20100514.bckp2-definitivo-final
 - Proyecto-????????
- Opción 2: VCS's "tradicionales": *CVS*, *Subversion*...
 - Requieren normalmente conexión a un repositorio central

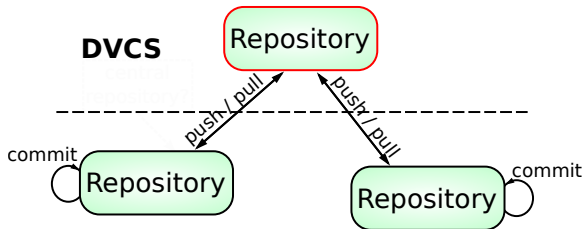
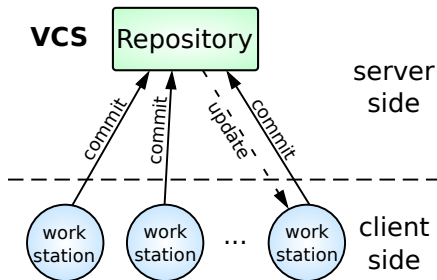
Gestión de cambios en un fichero, proyecto...

- Opción 0: no hacer nada (versión única)
- Opción 1: Sistemas rudimentarios (copias manuales)
 - Proyecto
 - Proyecto.bckp
 - Proyecto-20100514.bckp
 - Proyecto-20100514.bckp2
 - Proyecto-20100514.bckp2-final
 - Proyecto-20100514.bckp2-definitivo
 - Proyecto-20100514.bckp2-definitivo-final
 - Proyecto-????????
- Opción 2: VCS's "tradicionales": *CVS*, *Subversion*...
 - Requieren normalmente conexión a un repositorio central
- Opción 3: DVCS's: *Bazaar*, *Mercurial*, *Monotone*, **git** ...
 - Autonomía y flexibilidad **total** (i.e. *commits* locales)

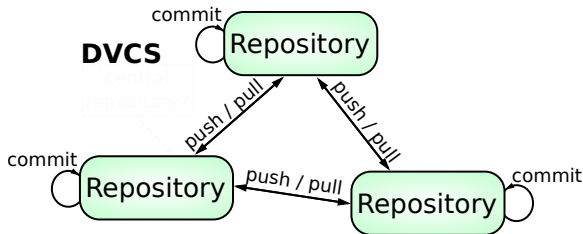
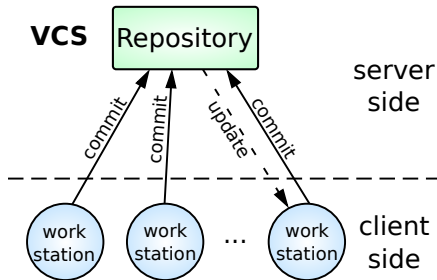
VCS vs DVCS



VCS vs DVCS



VCS vs DVCS



Ventajas de los DVCS

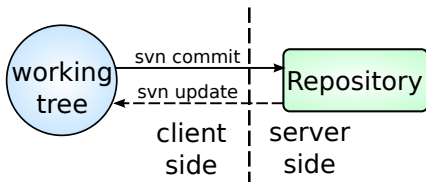
- Repositorio completo en local: nuevas posibilidades: histórico en local, gestión de ramas, *commits* offline...
- Mejora el *workflow* para trabajos colaborativos
- Facilita la integración de cambios de fuentes externas
- Facilita la gestión de distintas versiones de un proyecto
- Separación más “eficiente” entre trabajo privado y público
- Copias de seguridad implícitas
- No necesitan conexión para la mayoría de operaciones
- Y mucho más...

Desventajas de los DVCS

- Más complejos que los VCS. Curva de aprendizaje mayor
- Importación inicial más lenta (clonado del repositorio)
- Los clones completos pueden ocupar más espacio que una simple importación de la última revisión

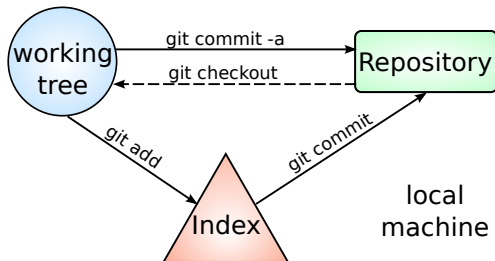
Subversion (VCS) vs Git (DVCS)

Subversion



Requires the two client / server parts to work

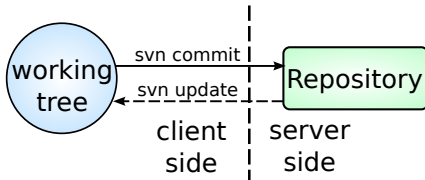
Git



Doesn't require anything else but the local machine itself

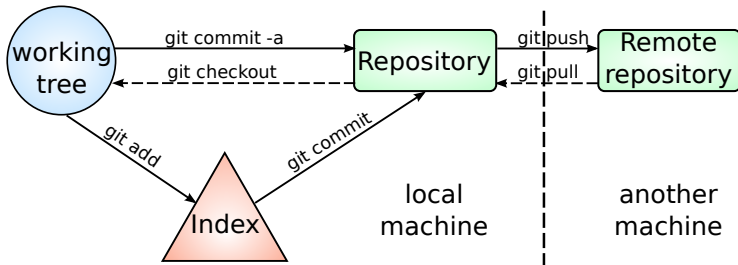
Subversion (VCS) vs Git (DVCS)

Subversion

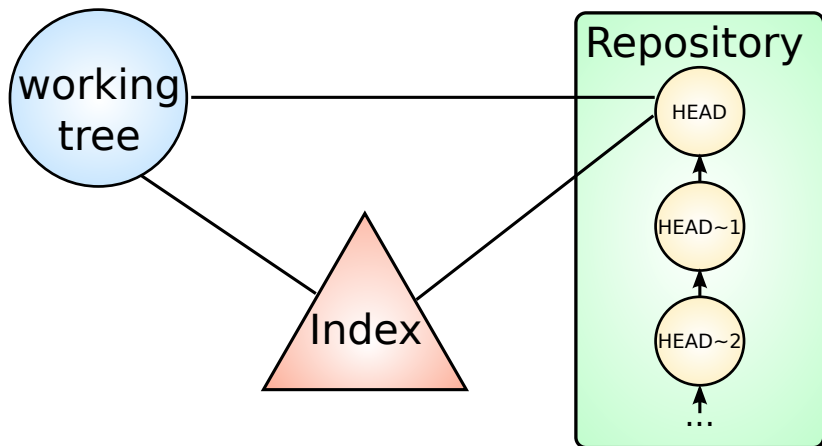


Requires the two client / server parts to work

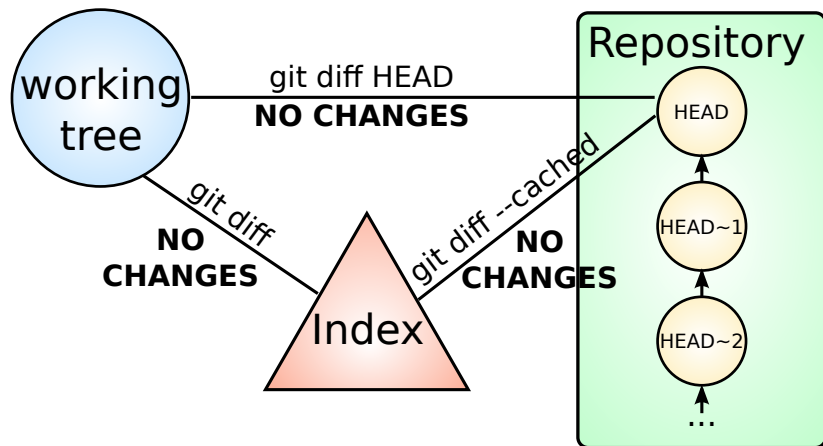
Git



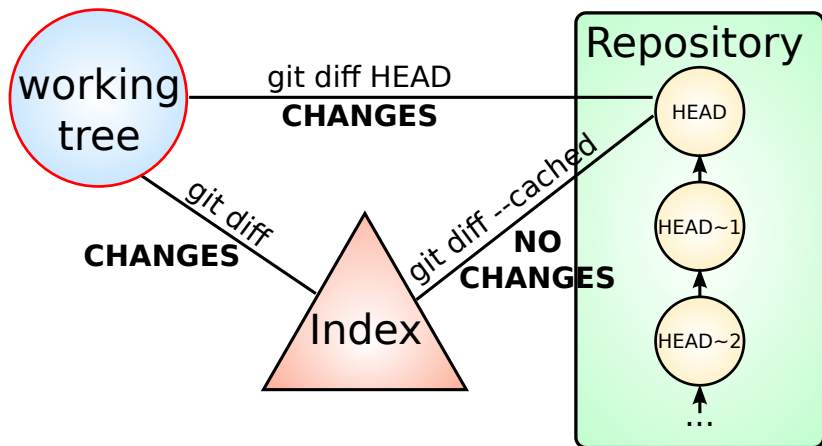
Workflow típico en git



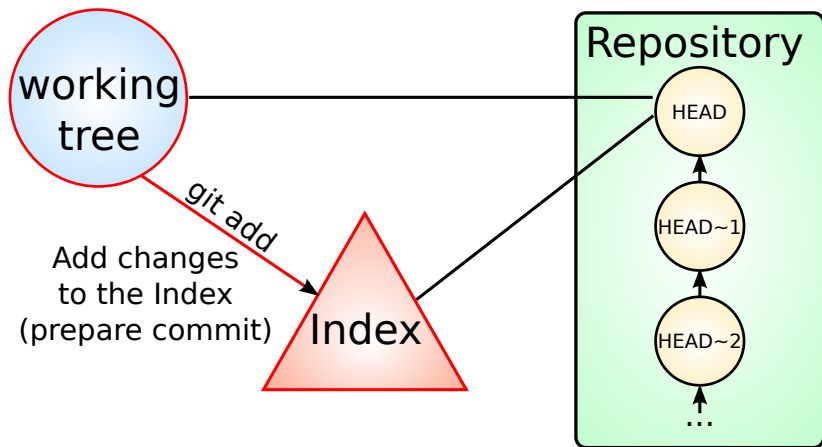
Workflow típico en git



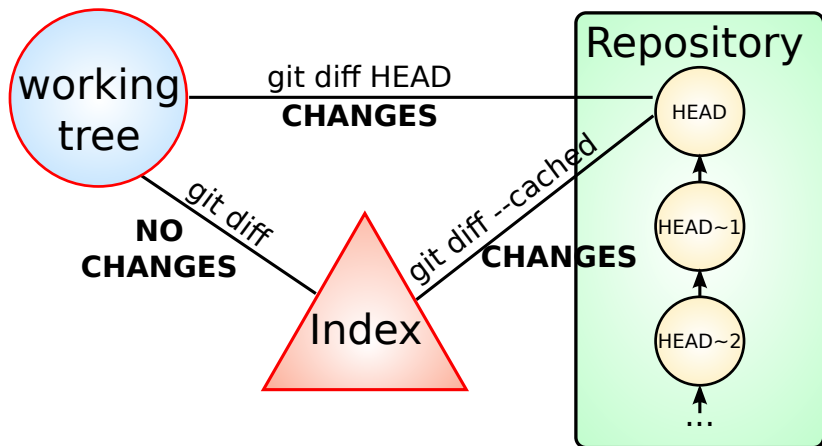
Workflow típico en git



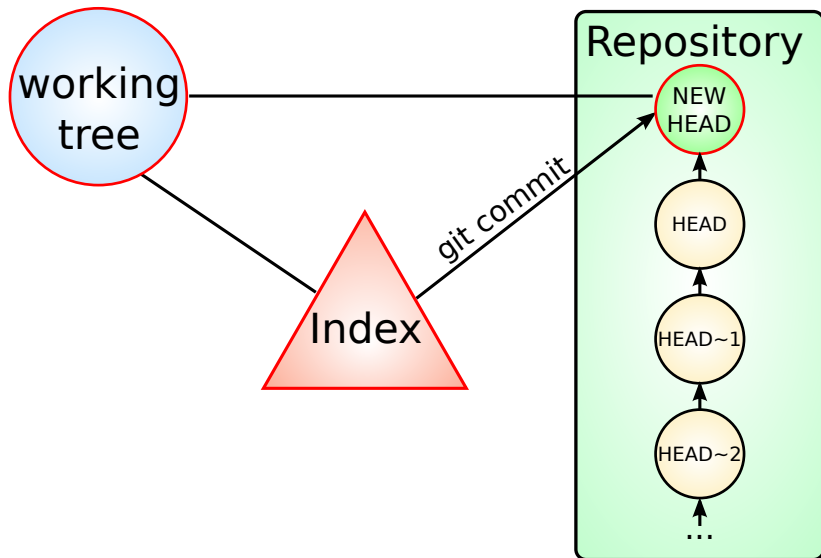
Workflow típico en git



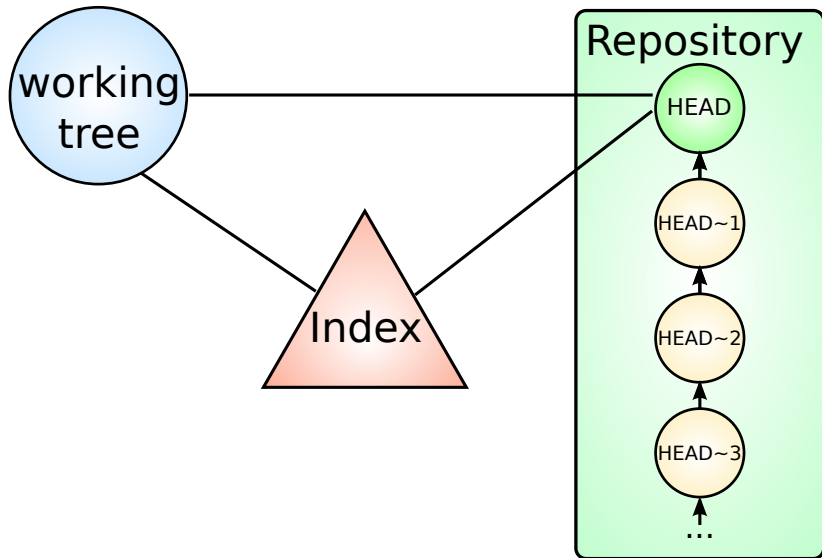
Workflow típico en git



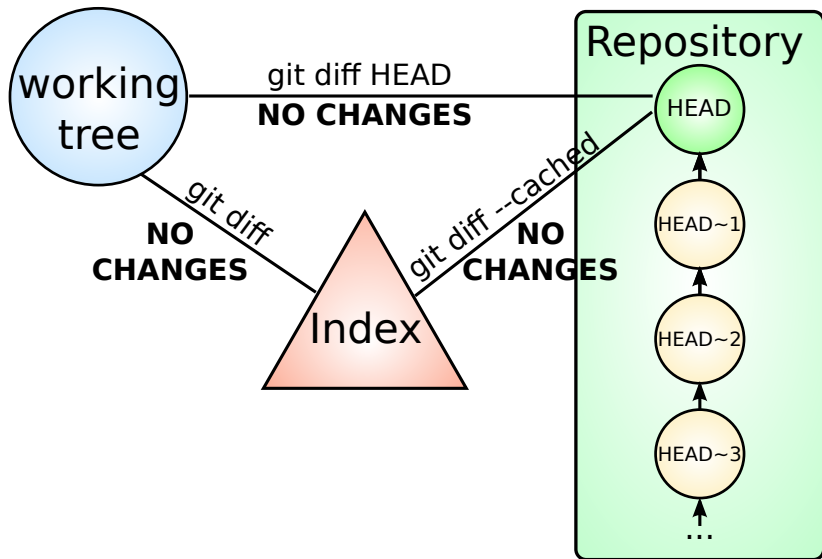
Workflow típico en git



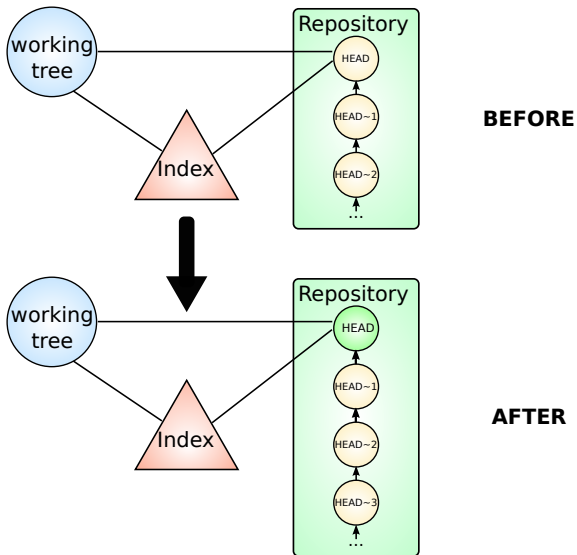
Workflow típico en git



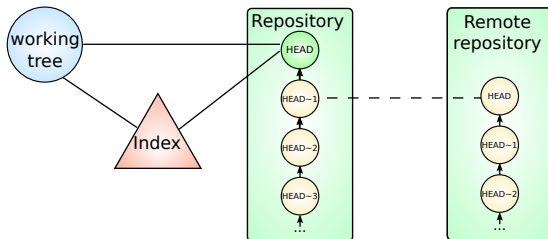
Workflow típico en git



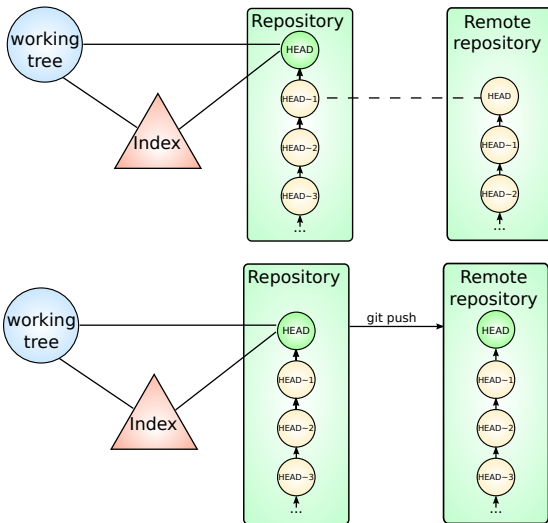
Workflow típico en git



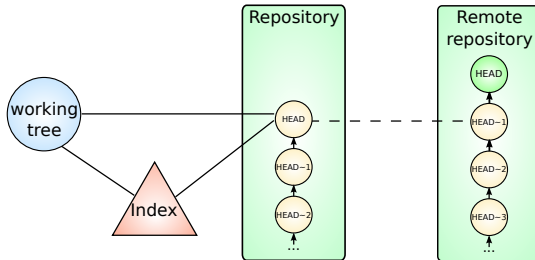
Workflow típico en git - git push



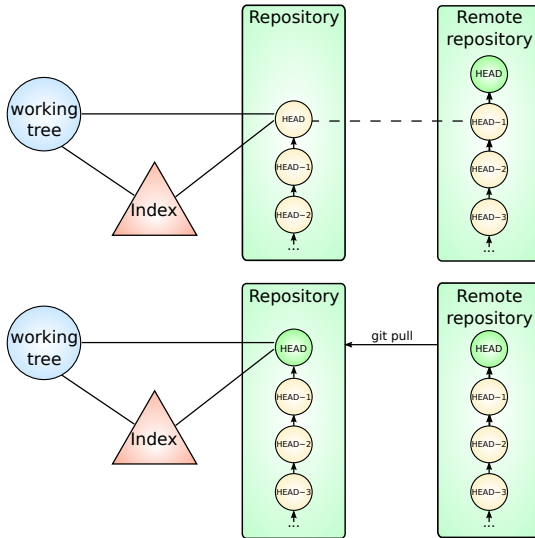
Workflow típico en git - git push



Workflow típico en git - git pull



Workflow típico en git - git pull



Workflow típico en git - Un ejemplo concreto

```
$ mkdir /tmp/git
$ cd /tmp/git

$ git init .                # Initialize repository
Initialized empty Git repository in /tmp/git/.git/

$ touch A B C
$ git add A B C             # Add new files to Index

$ git commit -m "Initial commit" # First commit!
[master (root-commit) fal6ae4] Initial commit
0 files changed, 0 insertions(+), 0 deletions(-)
create mode 100644 A
create mode 100644 B
create mode 100644 C
```

Workflow típico en git - Un ejemplo concreto

```
$ git diff --stat          # Working Tree <-> Index
$ git diff --stat HEAD     # Working Tree <-> HEAD
$ git diff --stat --cached # Index <-> HEAD
```

<<< MAKE SOME CHANGES IN THE WORKING TREE >>>

```
$ git diff --stat          # Working Tree <-> Index
A |      3 +++
C |      3 +++
2 files changed, 3 insertions(+), 0 deletions(-)
```

```
$ git diff --stat HEAD     # Working Tree <-> HEAD
A |      3 +++
C |      3 +++
2 files changed, 3 insertions(+), 0 deletions(-)
```

```
$ git diff --stat --cached # Index <-> HEAD
```


Workflow típico en git - Un ejemplo concreto

```
$ git add A C                                # Move changes to Index

$ git diff --stat                             # Working Tree <-> Index

$ git diff --stat HEAD                       # Working Tree <-> HEAD
A |      3 +++
C |      3 +++
2 files changed, 3 insertions(+), 0 deletions(-)

$ git diff --stat --cached                   # Index <-> HEAD
A |      3 +++
C |      3 +++
2 files changed, 3 insertions(+), 0 deletions(-)
```

Workflow típico en git - Un ejemplo concreto

```
$ git commit -m "My first patch :-)"    # Commit changes  
[master 614c4e2] My first patch :-)  
2 files changed, 3 insertions(+), 0 deletions(-)
```

```
$ git diff --stat                      # Working Tree <-> Index  
$ git diff --stat HEAD                 # Working Tree <-> HEAD  
$ git diff --stat --cached             # Index <-> HEAD
```

```
$ git show --stat HEAD                 # Show last commit's stats  
commit 614c4e224284e2719fe040b64685b790606000a6  
Author: Mario Sanchez Prada <msanchez@igalia.com>  
Date:   Mon May 3 09:31:19 2010 +0200
```

```
My first patch :-)
```

```
A |      3 +++  
C |      3 +++  
2 files changed, 3 insertions(+), 0 deletions(-)
```

Workflow típico en git - Un ejemplo concreto

```
$ git log                                # Show history up-to-date
commit 614c4e224284e2719fe040b64685b790606000a6
Author: Mario Sanchez Prada <msanchez@igalia.com>
Date:   Mon May 3 09:31:19 2010 +0200
```

My first patch :-)

```
commit fal6ae4ff64b3b968ca8991c90f123e62a6a009c
Author: Mario Sanchez Prada <msanchez@igalia.com>
Date:   Mon May 3 09:30:11 2010 +0200
```

Initial commit

Workflow típico en git (resumen)

Comandos básicos:

- `git init`: Inicializa el directorio para trabajar con *git*
- `git add`: Añade cambios en el *Working Tree* al *Index*
- `git commit`: Consolida en el repositorio los cambios del *Index*, creando un nuevo commit y actualizando el *HEAD*.
- `git diff`: Cambios entre el *Working Tree* y el *Index*
- `git push`: Envía cambios locales a un repositorio remoto.
- `git pull`: Trae cambios de un repositorio remoto al local.

Workflow típico en git (resumen)

Comandos básicos (versión extendida):

- `git init`: Inicializa el directorio para trabajar con *git*
- `git add`: Añade cambios en el *Working Tree* al *Index*
 - `git add -u`: Añade todos los cambios al *Index*
 - `git add -patch`: Pregunta interactivamente que cambios se quieren meter en el *Index* (por *hunks*)
- `git commit`: Consolida en el repositorio los cambios del *Index*, creando un nuevo commit y actualizando el *HEAD*.
 - `git commit -a:git add -u && git commit`
- `git diff`: Cambios entre el *Working Tree* y el *Index*
 - `git diff HEAD`: Cambios entre el *Working Tree* y el *HEAD*
 - `git diff -cached`: Cambios entre el *Index* y el *HEAD*
- `git push`: Envía cambios locales a un repositorio remoto.
- `git pull`: Trae cambios de un repositorio remoto al local.

Workflow típico en git (resumen)

Otros comandos (extremadamente) útiles:

- `git clone <URL>`: Clona un repositorio remoto
- `git rm <file>`: Elimina `file` y prepara el *Index* con los cambios para el próximo *commit*.
- `git mv <fileA> <fileB>`: Renombra `fileA` como `fileB` y prepara el *Index* con los cambios.
- `git log`: Muestra la historia (*log*) desde *HEAD*
- `git status`: Muestra el estado del *Working Tree* y el *Index*
- `git show <commitID>`: Visualiza el *commit* `commitID`

Workflow típico en git (resumen): git status

```
<<< RIGHT BEFORE ADDING A AND C TO INDEX WITH 'git add A C' >>>
```

```
$ git status
# On branch master
# Changed but not updated:
#   (use "git add <file>..." to update what will be committed)
#   (use "git checkout -- <file>..." to discard changes in working directory)
#
# modified:   A
# modified:   C
#
no changes added to commit (use "git add" and/or "git commit -a")
```

```
$ git add A
```

```
$ git status
# On branch master
# Changes to be committed:
#   (use "git reset HEAD <file>..." to unstage)
#
# modified:   A
#
# Changed but not updated:
#   (use "git add <file>..." to update what will be committed)
#   (use "git checkout -- <file>..." to discard changes in working directory)
#
# modified:   C
#
```

Workflow típico en git (resumen): git show

```
<<< RIGHT AFTER COMMITTING CHANGES WITH 'git commit' >>>
```

```
$ git show
commit 614c4e224284e2719fe040b64685b790606000a6
Author: Mario Sanchez Prada <msanchez@igalia.com>
Date:   Mon May 3 09:31:19 2010 +0200
```

```
    My first patch :-)
```

```
diff --git a/A b/A
index e69de29..1802a74 100644
--- a/A
+++ b/A
@@ -0,0 +1,3 @@
+aaa
+bbb
+ccc
diff --git a/C b/C
index e69de29..1802a74 100644
--- a/C
+++ b/C
@@ -0,0 +1,3 @@
+aaa
+bbb
+ccc
```


Comparación comandos Subversion - Git

Subversion	Git
svnadmin create <i>repo</i> svn import <i>filepath</i>	git init git add <i>filepath</i> git commit
svn checkout <i>URL</i>	git clone <i>URL</i>
svn add rm mv <i>file</i>	git add rm mv <i>file</i>
svn diff	git diff
svn commit	git commit -a git push
svn update	git pull
svn status	git status
svn log	git log
svn blame <i>file</i>	git blame <i>file</i>

Ramas y repositorios

¿Qué es una rama?

Una *rama* es una línea de trabajo paralela que nos permite realizar y probar ciertos cambios a partir de un punto dado sin alterar la línea de trabajo original desde la que fue creada.

¿Qué es una rama?

Una *rama* es una línea de trabajo paralela que nos permite realizar y probar ciertos cambios a partir de un punto dado sin alterar la línea de trabajo original desde la que fue creada.

Una vez que los cambios realizados en la *rama* han sido comprobados se pueden integrar en la rama original, resolviendo los “conflictos” que pudiesen haber surgido.

¿Qué es una rama?

Una *rama* es una línea de trabajo paralela que nos permite realizar y probar ciertos cambios a partir de un punto dado sin alterar la línea de trabajo original desde la que fue creada.

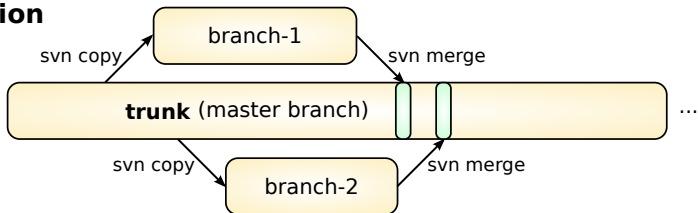
Una vez que los cambios realizados en la *rama* han sido comprobados se pueden integrar en la rama original, resolviendo los “conflictos” que pudiesen haber surgido.

Implementación:

- **En la mayoría de (D)VCS's:** Directorios separados
- **En *git* :** Un *alias* al commit que será el *HEAD* de esa rama (coste casi cero, muy rápido).

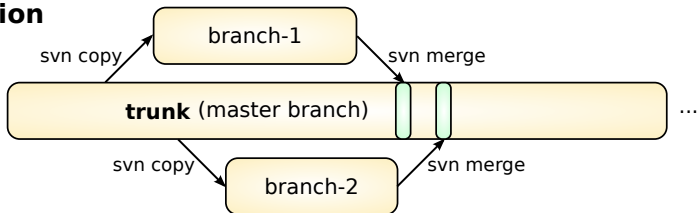
¿Qué es una rama? - Subversion vs Git

Subversion

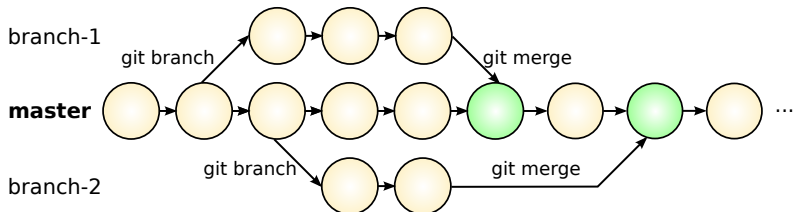


¿Qué es una rama? - Subversion vs Git

Subversion



Git



Ramas en git

- Crear una rama:

```
git branch <branch-name> [<start-point>]
```

- Borrar una rama:

```
git branch (-d | -D) <branch-name>
```

- Renombrar una rama:

```
git branch -m <old-name> <new-name>
```

- Listar las ramas actuales:

```
git branch [ -r | -a ]
```

- Situar en una rama:

```
git checkout [-b] <branch-name>
```


Ramas en git - Un ejemplo concreto

```
$ git branch
```

```
* master
```

```
$ git branch mybranch-1
```

```
$ git branch mybranch-2
```

```
$ git branch
```

```
* master
```

```
mybranch-1
```

```
mybranch-2
```

```
$ git checkout mybranch-1
```

```
Switched to branch 'mybranch-1'
```

```
$ git branch
```

```
master
```

```
* mybranch-1
```

```
mybranch-2
```

Ramas en git - Un ejemplo concreto

```
$ git branch -m mybranch-1 branch-1
```

```
$ git branch -m mybranch-2 branch-2
```

```
$ git branch
```

```
master
```

```
* branch-1
```

```
branch-2
```

```
$ git checkout master
```

```
Switched to branch 'master'
```

```
$ git branch -d branch-1
```

```
Deleted branch branch-1 (was 59f1309).
```

```
$ git branch -d branch-2
```

```
Deleted branch branch-2 (was e721bce).
```

```
$ git branch
```

```
* master
```

¿Por qué usar ramas?

- Son rápidas, cómodas y muy “baratas” :-)
- Para probar una idea (experimentación)
- Separar trabajo público de privado
- Una rama por *bug/feature*
- Preparar una nueva *release* (integración)
- Como *backup* temporal (*snapshot*)
- Es la forma habitual de trabajar en *git*.

Repositorios

Dos tipos de repositorios:

- **Repositorio local:** Directorio local de trabajo bajo control de versiones con *git* (contiene un directorio `.git/`).
En un repositorio local podemos tener múltiples ramas
Se “crean” mediante alguno de los siguientes pasos:
 - `git init`: Inicializa el directorio actual para ser utilizado con *git* (necesario hacer `git add <file>` y `git commit`)
 - `git clone <URL>`: Clona un repositorio remoto en un directorio local, manteniendo un “enlace” con el original a través del repositorio remoto `origin`.
- **Repositorios remotos:** Cualquier repositorio ajeno el cual tenemos clonado en nuestro repositorio local de trabajo
Se “crean” con `git remote add <name> <URL>`
`git clone <URL>` crea uno automáticamente: `origin`

Ramas locales y ramas remotas

Dos tipos de ramas:

- **Ramas locales:** Ramas creadas localmente para trabajar en ellas realizando los cambios que sean necesarios.
- **Ramas remotas:** Ramas de un repositorio remoto que podemos asociar a ramas locales para trabajar con ellas.

Aclaraciones:

Tanto las ramas **locales** como las **remotas** están siempre en nuestro **repositorio local**

Las ramas **remotas** actúan como una *cache* de ramas que son locales en **repositorios remotos**.

Las ramas **locales** se envían/actualizan (*push/pull*) a/desde los **repositorios remotos** a través de sus ramas **remotas** asociadas.

Ramas locales y remotas - Un ejemplo concreto

```
$ git branch -a
* master
  remotes/origin/master
  remotes/origin/other-branch
```

```
$ git branch private
```

```
$ git branch otherbranch origin/other-branch
Branch otherbranch set up to track
remote branch other-branch from origin.
```

```
$ git branch -a
* master
  private
  otherbranch
  remotes/origin/master
  remotes/origin/other-branch
```

Un ejemplo de ramas y repositorios: epiphany

Ramas locales

```
$ git branch
136292
539716
602547
608980
* 611400
alex
master
```

Uso de ramas en este ejemplo:

- Una rama por *bug*
- Una rama experimental (*alex*)
- Una rama “principal” (*master*).

Ramas remotas

```
$ git remote
origin

$ git branch -r
origin/HEAD -> origin/master
origin/Release043
origin/bookmarks-toolbars-changes
origin/bookmarksbar-separation
origin/css-fonts-branch
origin/eog-menu-api
origin/gnome-2-10
[...]
origin/master
origin/mozilla-embed-strings
origin/new-completion
origin/new-downloader
origin/pre-gnome-2-10
origin/pre-gnome-2-14
origin/pre-gnome-2-6
origin/pre-gnome-2-8
origin/release-1-2-1
origin/webcore-branch
origin/webkit
origin/xulrunner
```

Actualizando las ramas

Dos situaciones típicas:

- **Añadir** a una **rama pública** cambios presentes en una **rama privada** (integrar cambios locales)

- **Modificar** una **rama privada** con código nuevo presente en **rama pública** desde la que fue creada (actualizar)

Actualizando las ramas

Dos situaciones típicas:

- **Añadir** a una **rama pública** cambios presentes en una **rama privada** (integrar cambios locales)
 - No podemos alterar la historia pasada de la rama pública
 - Necesitamos cambios “limpios” listos para ser aplicados a partir del *HEAD* de la rama pública
- **Modificar** una **rama privada** con código nuevo presente en **rama pública** desde la que fue creada (actualizar)

Dos situaciones típicas:

- **Añadir** a una **rama pública** cambios presentes en una **rama privada** (integrar cambios locales)
 - No podemos alterar la historia pasada de la rama pública
 - Necesitamos cambios “limpios” listos para ser aplicados a partir del *HEAD* de la rama pública
- **Modificar** una **rama privada** con código nuevo presente en **rama pública** desde la que fue creada (actualizar)
 - Podemos alterar la historia de la rama privada
 - Resulta más interesante “cambiar el pasado” siempre que sea necesario para tener siempre “cambios limpios” desde la rama desde la cual fue creada (pudo haber cambiado)

Actualizando las ramas - git merge

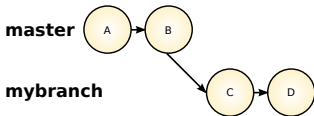
Actualizar una rama pública: `git merge <branch>`

- No hay **cambios** en la rama pública desde que se creo la privada:
- Hay **cambios** en la rama pública desde que se creo la privada:

Actualizando las ramas - git merge

Actualizar una rama pública: `git merge <branch>`

- **No** hay **cambios** en la rama pública desde que se creo la privada:

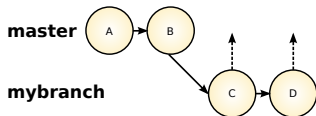


- **Hay cambios** en la rama pública desde que se creo la privada:

Actualizando las ramas - git merge

Actualizar una rama pública: `git merge <branch>`

- **No** hay **cambios** en la rama pública desde que se creo la privada:

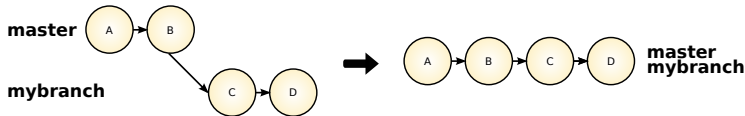


- **Hay cambios** en la rama pública desde que se creo la privada:

Actualizando las ramas - git merge

Actualizar una rama pública: `git merge <branch>`

- **No** hay **cambios** en la rama pública desde que se creo la privada:

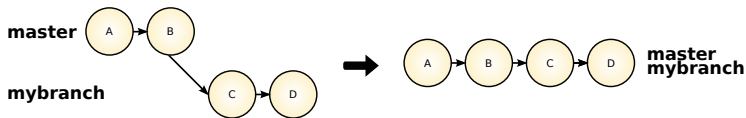


- **Hay** **cambios** en la rama pública desde que se creo la privada:

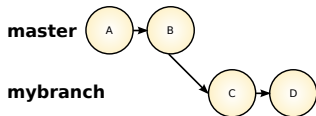
Actualizando las ramas - git merge

Actualizar una rama pública: `git merge <branch>`

- No hay **cambios** en la rama pública desde que se creo la privada:



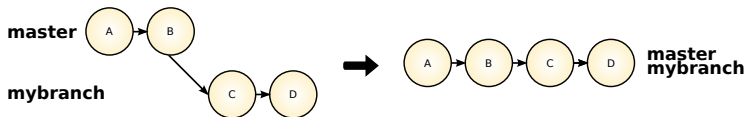
- Hay **cambios** en la rama pública desde que se creo la privada:



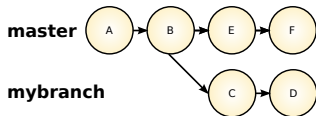
Actualizando las ramas - git merge

Actualizar una rama pública: `git merge <branch>`

- **No hay cambios** en la rama pública desde que se creo la privada:



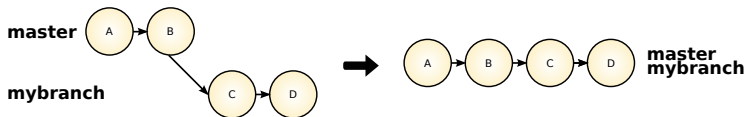
- **Hay cambios** en la rama pública desde que se creo la privada:



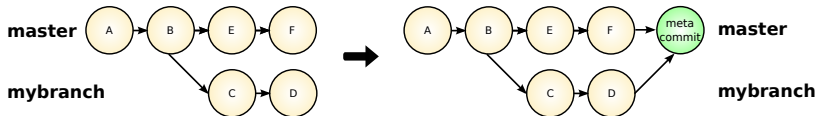
Actualizando las ramas - git merge

Actualizar una rama pública: `git merge <branch>`

- No hay **cambios** en la rama pública desde que se creo la privada:

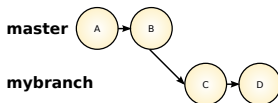


- Hay **cambios** en la rama pública desde que se creo la privada:



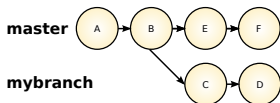
Actualizando las ramas - git rebase

Actualizar una rama privada: `git rebase <new-base>`



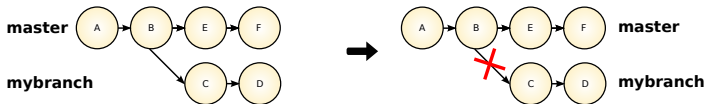
Actualizando las ramas - git rebase

Actualizar una rama privada: `git rebase <new-base>`



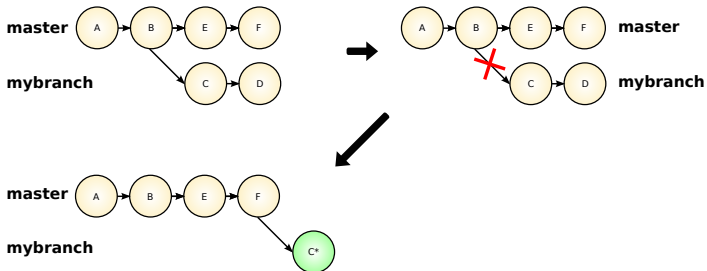
Actualizando las ramas - git rebase

Actualizar una rama privada: `git rebase <new-base>`



Actualizando las ramas - git rebase

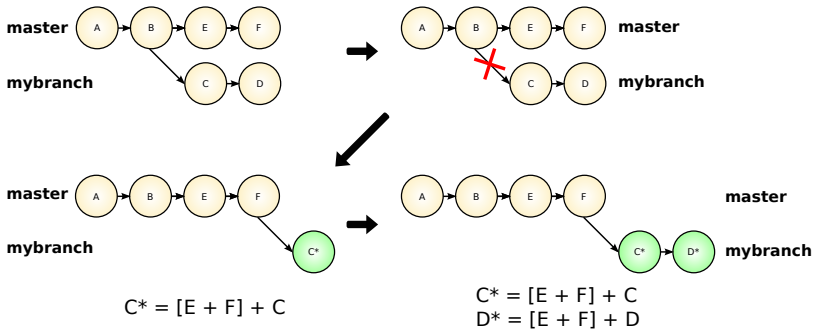
Actualizar una rama privada: `git rebase <new-base>`



$$C^* = [E + F] + C$$

Actualizando las ramas - git rebase

Actualizar una rama privada: `git rebase <new-base>`



Consejo (siempre que sea posible)

Antes de actualizar una rama pública con `git merge` para luego hacer un `git push`, hacer primero un `git rebase` desde la rama privada, para evitar la aparición del *metacommit*.

```
$ git checkout mybranch # Change to private branch  
  
$ git rebase master      # Rebase private branch  
  
$ git checkout master    # Change back to master  
  
$ git merge mybranch     # Merge changes from mybranch  
  
$ git push               # Push to remote repository
```

Otras cosillas...

Otros comandos interesantes

- Crear una etiqueta:
`git tag <tag-name>`
- Revertir un *commit* (crea un nuevo *commit* sobre *HEAD*):
`git revert <commitID>`
- Copiar un commit concreto sobre *HEAD*:
`git cherry-pick <commitID>`
- Mostrar los autores de los últimos cambios en un fichero:
`git blame <filepath>`
- Aplicar un *diff* en el *Working Tree* (no crea un *commit*):
`git apply <diff-file>`
- Crear parches “completos” desde un punto dado:
`git format-patch <base-commitID>`
- Aplicar parches “completos” (manteniendo autor, log...):
`git am <patchfile>`

Para investigar con más calma

- Guardar y restaurar datos temporales:

```
git stash & git stash apply
```

- Modificar el *HEAD*, el *Index* y el *Working Tree*:

```
git reset [ -soft | -hard ] <commitID>
```

- Reescribir el pasado manualmente (cirugía de *commits*):

- `git rebase -i <commitID>`

- `git commit -amend`

- Reescribir el pasado de forma automática:

```
git filter-branch <params>
```

- Consultar el histórico (y restaurar datos):

```
git reflog
```

- Enviar parches por correo:

```
git send-email <email-address>
```

- Integración con *Subversion*:

```
git svn <command> [ options ] [ arguments ]
```

Donde ir a partir de aquí

- ***Git from the bottom up (Excelente tutorial):***
<http://ftp.newartisans.com/pub/git.from.bottom.up.pdf>
- ***Git User's Manual:***
<http://www.kernel.org/pub/software/scm/git/docs/v1.7.1/user-manual.html>
- ***Git tutorial manual page (man gittutorial):***
<http://www.kernel.org/pub/software/scm/git/docs/v1.7.1/gittutorial.html>
- ***Everyday GIT With 20 Commands Or So:***
<http://www.kernel.org/pub/software/scm/git/docs/v1.7.1/everyday.html>
- ***Gitorious: infrastructure for open source projects with Git:***
<http://gitorious.org>

¿Preguntas?

Descarga esta presentación en:

<http://tinyurl.com/20100515-gititdone>

¡Gracias!

Descarga esta presentación en:

`http://tinyurl.com/20100515-gititdone`